

Teilabgabe 2

Anforderungen

1. Funktionale Anforderungen

Erweitern Sie die im Rahmen der ersten Teilabgabe entwickelte To-Do-Anwendung um eine zweite API auf Basis von GraphQL sowie um eine dokumentenbasierte Persistenz mit MongoDB. Ziel dieser Teilabgabe ist es, typische Staerken flexibler Abfragen, verschachtelter Datenmodelle und ereignisbasierter Aktualisierungen in einer Web-Anwendung praktisch umzusetzen.

Welche funktionalen Anforderungen Ihre Loesung dabei mindestens erfuellen soll, ist in den nachfolgenden Punkten aufgefuehrt.

1. To-Do-Eintraege muessen neben den bereits vorhandenen Basisfeldern um weitere strukturierte Informationen, die allerdings nicht unbedingt immer für alle To-Dos Pflichtangaben sein sollen, zum Beispiel:
 - Tags
 - Checklisten
 - Subtasks
 - Kommentare
 - Prioritaet
 - Faelligkeitsdatum
 - Bearbeitungsverlauf
2. Die UI muss unterschiedliche Sichten auf die To-Do-Eintraege unterstuetzen, zum Beispiel eine Uebersicht mit nur Titel und Status, eine Detailansicht mit allen Informationen oder eine Filteransicht, die nur To-Dos mit bestimmten Tags oder Prioritaeten anzeigt. Alle diese Informationen muessen ueber die GraphQL-API bereitgestellt und flexibel abfragbar sein.
3. Die Anwendung ist als Multi-User-System zu verstehen, in dem mehrere Nutzerinnen und Nutzer gleichzeitig mit denselben To-Do-Eintraegen arbeiten koennen. Aenderungen an To-Do-Eintraegen muessen in Echtzeit an verbundene Clients uebermittelt werden koennen. Dazu zaehlen insbesondere:
 - neu erstellte To-Do-Eintraege,
 - Statusaenderungen,
 - neu hinzugefuegte Kommentare oder Subtasks,
 - geloeschte oder archivierte Eintraege.

2. Technische Anforderungen

1. Backend
 - Node.js (mit Express.js oder einem vergleichbaren Framework).
 - MongoDB als dokumentenbasierte Datenbank.
 - Das Datenmodell soll dokumentenorientiert entworfen werden. Verschachtelte oder eingebettete Datenstrukturen sollen dort eingesetzt werden, wo dies fachlich sinnvoll ist, zum Beispiel fuer Kommentare, Tags oder Subtasks.

2. API

- GraphQL als API-Technologie mit:
 - Queries fuer flexible Lesezugriffe,
 - Mutations fuer Erstellen, Aendern und Loeschen,
 - Subscriptions fuer Realtime-Aktualisierungen.
- Neue To-Do-Eintraege sowie Kommentare, Tags oder Subtasks muessen ueber die GraphQL-API erstellt, veraendert und geloescht werden koennen. GraphQL-API ersetzt jetzt die REST-API als Hauptschnittstelle fuer die Anwendung.
- Die GraphQL-API muss es ermöglichen, genau die Datenfelder abzufragen, die für eine bestimmte Ansicht oder Funktionalität benötigt werden. Kein overfetching oder underfetching von Daten soll auftreten.

4. Frontend

- Das bestehende Frontend ist so zu erweitern, dass es die GraphQL-API nutzt.
- Mindestens eine Ansicht soll von GraphQL Subscriptions Gebrauch machen und Aenderungen ohne manuelles Neuladen darstellen.

Die optische Gestaltung der Benutzeroberflaeche ist weiterhin von nachrangiger Bedeutung; entscheidend sind vor allem die Funktionalitaet, eine nachvollziehbare Architekturentscheidung und eine saubere technische Umsetzung.

Im Rahmen dieser Teilabgabe muessen Authentifizierung und Autorisierung zunaechst nicht umgesetzt werden. Konzentrieren Sie sich zuerst auf die fachliche Funktionalitaet sowie auf die technische Struktur der GraphQL-API und der Realtime-Kommunikation.

3. Software-Engineering-Anforderungen

- Entwurf eines GraphQL-Schemas mit geeigneten Typen, Queries, Mutations und Subscriptions.
- Nachvollziehbare Begrueendung des dokumentenbasierten Datenmodells.
- Saubere Trennung von Resolvern, Geschaeftslogik und Persistenzzugriff.
- Tests fuer zentrale Teile der GraphQL-API, insbesondere fuer:
 - Queries,
 - Mutations,
 - Subscriptions.
- Dokumentation der Unterschiede zur ersten Teilabgabe (REST + SQL) und kurze Reflexion darueber, welche Anforderungen durch GraphQL und MongoDB besser oder schlechter abgebildet werden koennen.

4. Abgabe

Reichen Sie die folgenden Bestandteile Über README als Gruppe ein:

1. Ausführbarer Code als Git-Repository. Das Repository soll die gesamte Codebasis enthalten, einschliesslich Backend, Frontend, GraphQL-Schema, Datenbankbindung und Tests.
2. Dokumentation des GraphQL-Schemas, einschliesslich der unterstützten Queries, Mutations und Subscriptions.
3. Ausführbare Testing-Skripte.
4. Eine README.md als Teil des Codes, die Hinweise zur Installation, Nutzung und zum Testen enthaelt.
5. Eine kurze technische Reflexion (ca. 1 bis 2 Seiten oder aehnlicher Umfang) mit folgenden Punkten:
 - Begründung der Modellierung in MongoDB,
 - Rolle von GraphQL gegenüber der REST-API aus Teilabgabe 1,
 - Nutzen und Grenzen von Pub/Sub und Subscriptions in Ihrer Loesung.

Ziel dieser Teilabgabe ist nicht nur die technische Erweiterung der ersten Loesung, sondern auch der bewusste Vergleich unterschiedlicher API- und Persistenzparadigmen.